End-to-End Efficiency of a Packed Projected W1A1/N1 ResNet-18

Reviewer-response insert: BF16-versus-GGM latency, throughput, energy, persistent state, CUDA memory, projection accounting, and retained floating-point operations

Reviewer question. "Provide actual efficiency measurements (latency, throughput, memory, energy, or bit-ops) for a representative CNN, accounting for projection expansion and all retained full-precision modules. This is the decisive item."

Response. For complete ResNet-18 inference on an NVIDIA A40 GPU at batch size 1, the packed GGM custom CUDA kernel is **2.50× faster on CIFAR-10** and **2.24× faster on ImageNet** than the cuDNN BF16 baseline. It raises throughput from **613.85** to **1,532.51 images/s** on CIFAR-10 and from **609.67** to **1,365.81 images/s** on ImageNet. Gross GPU-board energy per image falls by **47.5%** and **23.7%**, respectively. Registered persistent tensor state falls by **74.5%** on CIFAR-10 and **71.3%** on ImageNet, while peak CUDA allocation falls by **41.3%** and **20.7%**. The complete forward path includes activation projection, sign conversion, bit packing, XOR/popcount reduction, BF16 boundaries, BatchNorm, residual additions, ReLU, average pooling, and the classifier.

Evaluated topology and implementation boundary. Both implementations use ResNet-18 at batch size 1. The comparison is between a conventional cuDNN BF16 ResNet-18 and an equivalence-preserving packed GGM implementation using a custom CUDA kernel. The GGM model uses channelwise random projection followed by binary sign comparison in place of conventional dense convolution. The reported packed backend exactly matched the regular PyTorch BF16 GGM implementation in the separate acceptance test.

Component	cuDNN BF16 baseline	Packed GGM runtime treatment
Convolutional layers	Dense BF16 cuDNN convolutions	FP32 activation projection, sign conversion, packed binary comparison, and popcount reduction
Projected weights	Conventional BF16 convolution weights	Preprojected and prepacked binary weight representation
BatchNorm	Standard BF16 inference BatchNorm	Fused into the packed reduction epilogue with BF16-equivalent boundaries
Residual additions	Separate BF16 tensor operation	Fused into the custom CUDA epilogue where applicable
ReLU	Separate BF16 activation	Fused into the custom CUDA epilogue where applicable
Pooling / classifier	Standard floating-point operators	Retained floating-point boundary and included end to end
Persistent state	Registered BF16 parameters and buffers	Compressed projected/packed state plus small floating-point boundaries
Temporary state	cuDNN workspaces and activations	Packed activation workspace, output cache, and intermediate CUDA state

Projection accounting. The benchmark measures the complete model call. Therefore, all activation projection, sign conversion, packing, packed-weight access, XOR, population count, BF16 conversion, fused BatchNorm, residual addition, ReLU, pooling, and classifier operations actually executed by the GGM implementation are charged to the reported latency, throughput, energy, and CUDA-memory results.

Headline protocol. NVIDIA A40 GPU; CUDA 12.1; PyTorch with CUDA 12.1 support; batch size 1; inference mode; 100 warm-up iterations; 500 timed iterations; ten timing repeats; NVIDIA NVML total-energy counter for gross board energy; three one-second active-energy repetitions; idle power measured over three two-second repetitions; and complete end-to-end ResNet-18 forward passes.

CIFAR-10 latency and throughput.

Model	Latency / image	Throughput	Relative result
cuDNN BF16 ResNet-18	1.6291 ms	613.85 img/s	1.00×
Packed GGM custom CUDA kernel	0.6525 ms	1,532.51 img/s	2.50×
GGM change	59.9% lower	149.7% higher	—

ImageNet latency and throughput.

Model	Latency / image	Throughput	Relative result
cuDNN BF16 ResNet-18	1.6402 ms	609.67 img/s	1.00×
Packed GGM custom CUDA kernel	0.7322 ms	1,365.81 img/s	2.24×
GGM change	55.4% lower	124.0% higher	—

GPU-board energy accounting.

Dataset	Model	Gross energy	Dynamic energy	Interpretation
CIFAR-10	cuDNN BF16 ResNet-18	147.788 mJ/image	22.084 mJ/image	Baseline
CIFAR-10	Packed GGM custom CUDA kernel	77.663 mJ/image	27.170 mJ/image	47.5% lower gross energy
ImageNet	cuDNN BF16 ResNet-18	168.474 mJ/image	41.914 mJ/image	Baseline
ImageNet	Packed GGM custom CUDA kernel	128.539 mJ/image	71.896 mJ/image	23.7% lower gross energy

Gross energy measures total GPU-board energy consumed per completed image during the active interval. Dynamic energy subtracts the measured idle-power baseline. The packed GGM kernel lowers gross energy per image on both datasets because inference completes substantially sooner, but its idle-corrected dynamic energy is higher, indicating greater incremental board power while the optimized kernel is active.

Persistent tensor-state and serialized-state accounting.

Dataset	Model	Registered state	Serialized state	Runtime cache	Peak CUDA
CIFAR-10	cuDNN BF16 ResNet-18	21.33 MiB	21.37 MiB	0.00 MiB	31.11 MiB
CIFAR-10	Packed GGM custom CUDA kernel	5.44 MiB	5.49 MiB	3.06 MiB	18.27 MiB
ImageNet	cuDNN BF16 ResNet-18	22.31 MiB	22.35 MiB	0.00 MiB	35.13 MiB
ImageNet	Packed GGM custom CUDA kernel	6.41 MiB	6.46 MiB	8.95 MiB	27.85 MiB

The packed GGM model replaces most conventional convolutional weight storage with projected and packed binary representations. The runtime cache contains reusable packed-activation workspace and output buffers. Peak CUDA allocation includes the registered model state, inputs, activations, cached workspaces, and temporary tensors reached during the complete forward pass.

Reviewer question. "Which ResNet-18 modules are quantized, and how much do retained full-precision components contribute? Does the gain survive realistic projection and floating-point accounting?"

Response. The packed GGM implementation replaces the principal convolutional computation in the ResNet-18 body with channelwise FP32 random projection, sign extraction, bit packing, and packed XOR/popcount reduction. BatchNorm, residual addition, and ReLU are fused into the custom CUDA epilogue where applicable, while the input/output BF16 boundaries, pooling, and classifier remain floating point. All projection, packing, fusion, residual, pooling, and classifier costs are included in the end-to-end measurements. Despite this complete accounting, the packed implementation remains **2.50× faster on CIFAR-10** and **2.24× faster on ImageNet**.

Code-level binary and floating-point map. Projected convolutional weights are prepared ahead of inference, converted to signs, and packed into binary words. During each forward pass, the custom CUDA path projects BF16 activations in FP32, applies the sign function, packs the resulting binary codes, compares them against the prepacked projected weights using XOR, and accumulates population counts. The integer mismatch result is converted back to the BF16 GGM output domain.

To preserve the output of the regular PyTorch BF16 GGM network, the packed epilogue maintains the required BF16 rounding boundaries around the raw GGM output, BatchNorm, and optional residual addition. ReLU is applied after the residual boundary. These operations are fused to avoid writing and rereading unnecessary intermediate activation tensors.

The following operations remain floating point and are included in the reported measurements:

- the BF16 input and output boundaries;
- FP32 activation projection before sign extraction;
- BatchNorm arithmetic as represented by the equivalence-preserving fused epilogue;
- residual additions at BF16-equivalent boundaries;
- the specialized or conservative stem boundary where applicable;
- global average pooling; and
- the final floating-point classifier.

Runtime component	GGM implementation treatment	Accounting status
Activation projection	FP32 channelwise random projection inside the CUDA path	Included end to end
Sign conversion and packing	Binary activation-code generation and machine-word packing	Included end to end
Convolutional reduction	XOR and population count against prepacked projected weights	Included end to end
BatchNorm	Fused equivalence-preserving BF16 epilogue	Included end to end
Residual addition	Fused where applicable with explicit BF16 boundary	Included end to end
ReLU	Fused after the residual or BatchNorm boundary	Included end to end
Pooling	Retained floating-point global average pooling	Included end to end
Classifier	Retained floating-point output layer	Included end to end
Runtime workspace	Packed activation and reusable output caches	Included in cache and peak CUDA memory

Contribution of retained floating-point work. The available measurements establish complete end-to-end performance but do not provide a kernel-by-kernel CUDA-time attribution. Therefore, the exact percentage contribution of projection, pooling, the classifier, and other retained floating-point operations cannot be reported from the current data alone. Their total cost is nevertheless already included in every reported latency, throughput, energy, and peak-memory result.

Does the gain survive realistic accounting? Yes, at the measured batch size of 1. The packed GGM implementation remains more than twofold faster on both datasets after charging the full model for FP32 projection, sign conversion, bit packing, XOR/popcount reduction, BF16 boundary handling, fused BatchNorm, residual operations, ReLU, pooling, classifier execution, runtime caches, and temporary CUDA allocations.

Current measurement boundary. The available benchmark covers batch size 1 only and does not include a CUDA operator profile, kernel occupancy analysis, or a batch-size sweep. Consequently, no claim is made here about scaling beyond batch size 1 or the percentage runtime contribution of individual CUDA kernels.

Combined result.

Dataset	Latency speedup	Throughput increase	Gross energy reduction	State reduction	Serialized reduction	Peak CUDA reduction
CIFAR-10	2.50×	149.7%	47.5%	74.5%	74.3%	41.3%
ImageNet	2.24×	124.0%	23.7%	71.3%	71.1%	20.7%